

Memory.

A scenario.

MAR started PyTorch

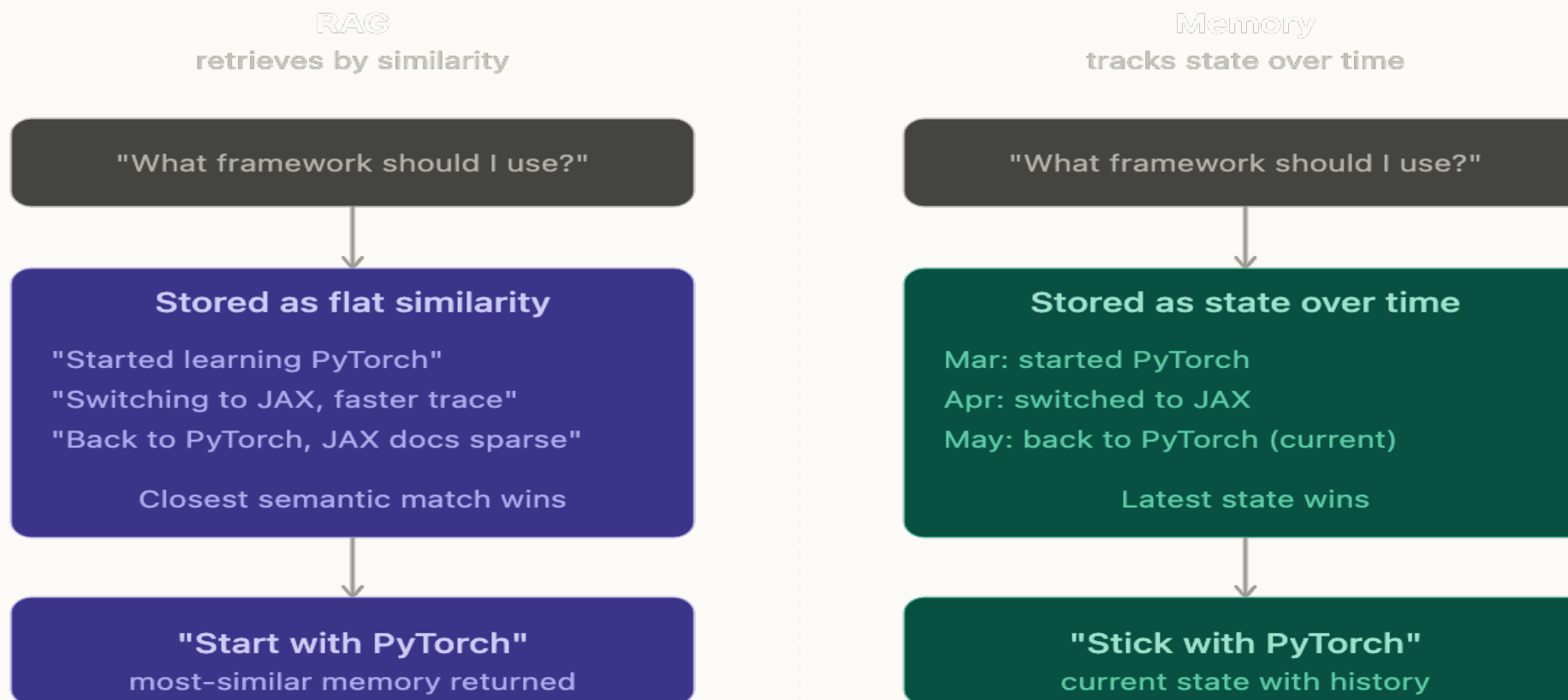
APR switched to JAX

MAY back to PyTorch

"What framework should I use?"

What should it say?

RAG versus memory.



RAG retrieves by similarity. Memory tracks state over time.

Two problems, one word.

- "What do I know."
- "What do I remember about you."
- The boundary between them is fuzzy.

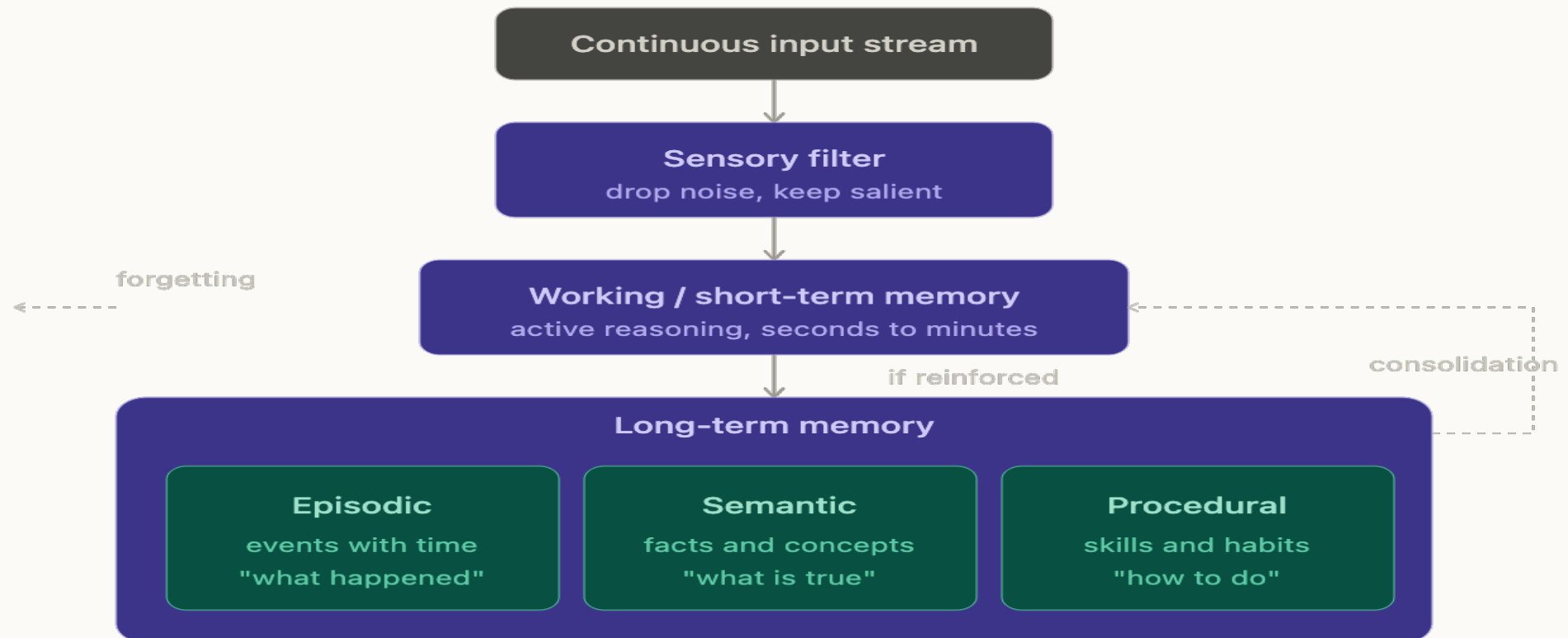
Three types of memory.

Episodic — *events*

Semantic — *facts*

Procedural — *skills*

Human memory pipeline.



Sensory → working → long-term (episodic, semantic, procedural).

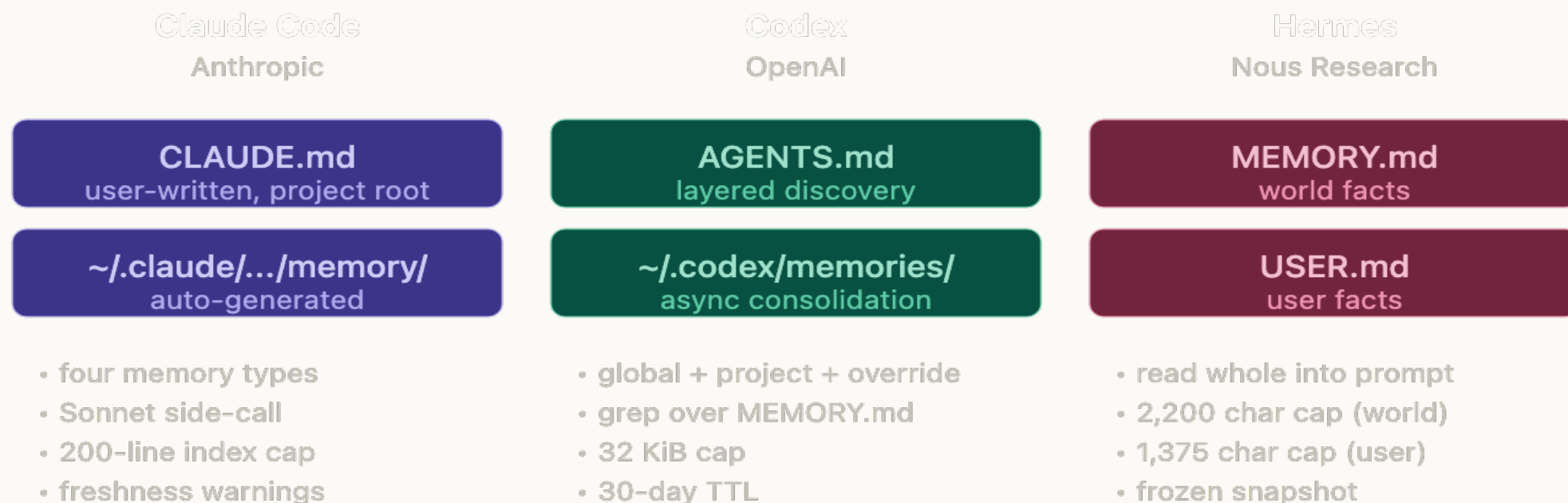
Ship memory Monday.

How would you do it?

A file.

- Plain markdown.
- Load at session start.
- Save at session end.

Files on disk — same primitive, three companies.



Same primitive: plain markdown, load at session start, no embeddings

Plain markdown. Load at session start. No embeddings.

Same idea, three companies.

Claude Code

→ CLAUDE.md + auto-memory

Codex

→ AGENTS.md layered

Hermes

→ MEMORY.md + USER.md

What they don't do.

- No embeddings.
 - No vector store.
 - Relevance via grep, or an LLM side-call.
-

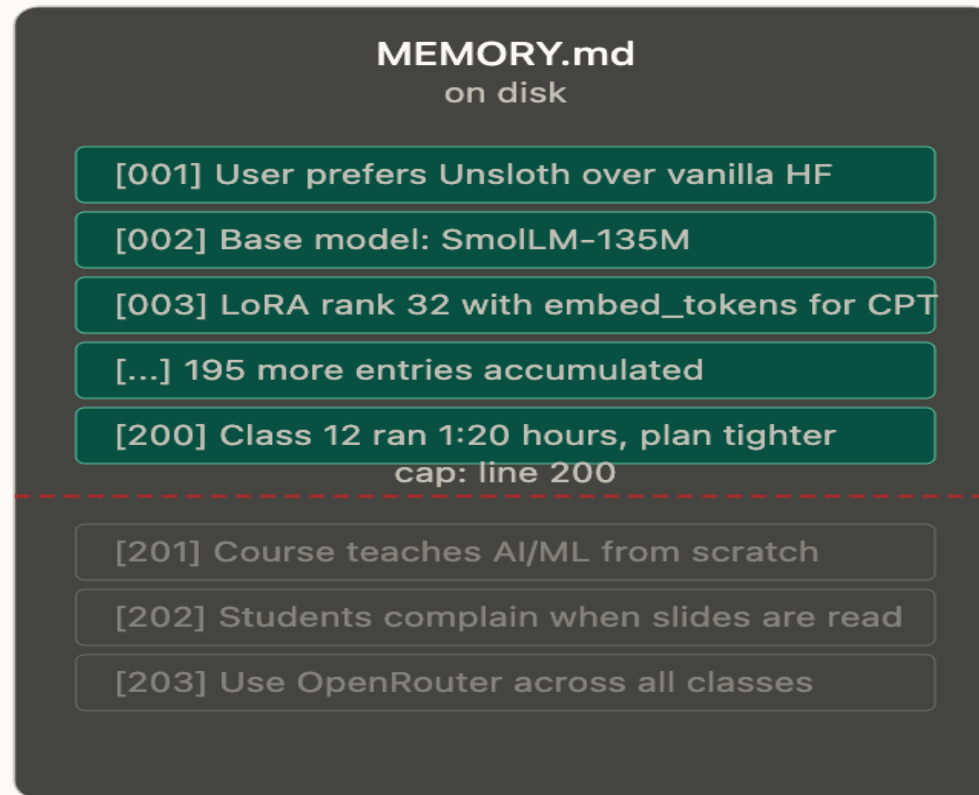
Three failure modes.

- 01 **The cap** — *size limit*
- 02 **Semantic miss** — *exact-match retrieval*
- 03 **Temporal & portability** — *no time, no team*

The cap.

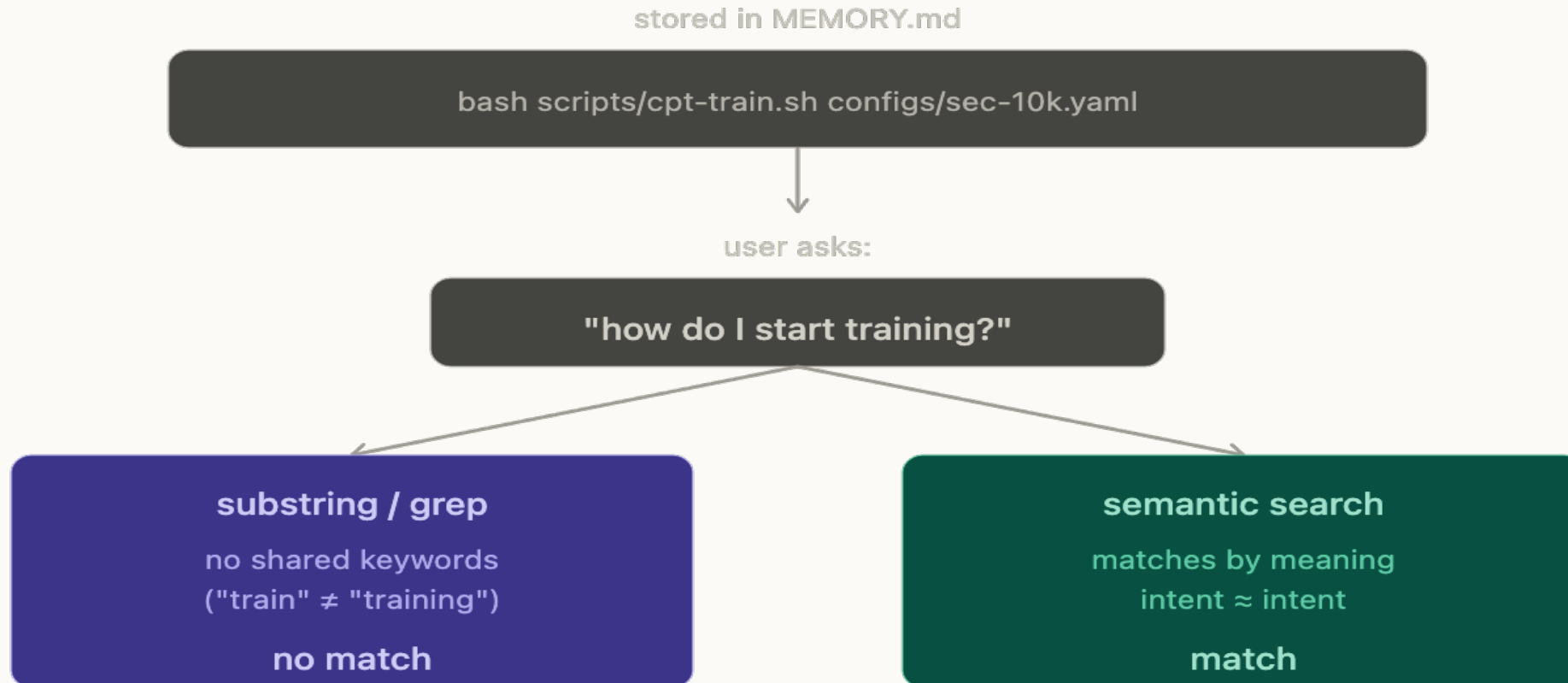
loaded into
system prompt

on disk,
not loaded



The hard ceiling — anything after line 200 stays on disk, unread.

Semantic miss.



"how do I start training?" vs. bash scripts/cpt-train.sh — no overlap.

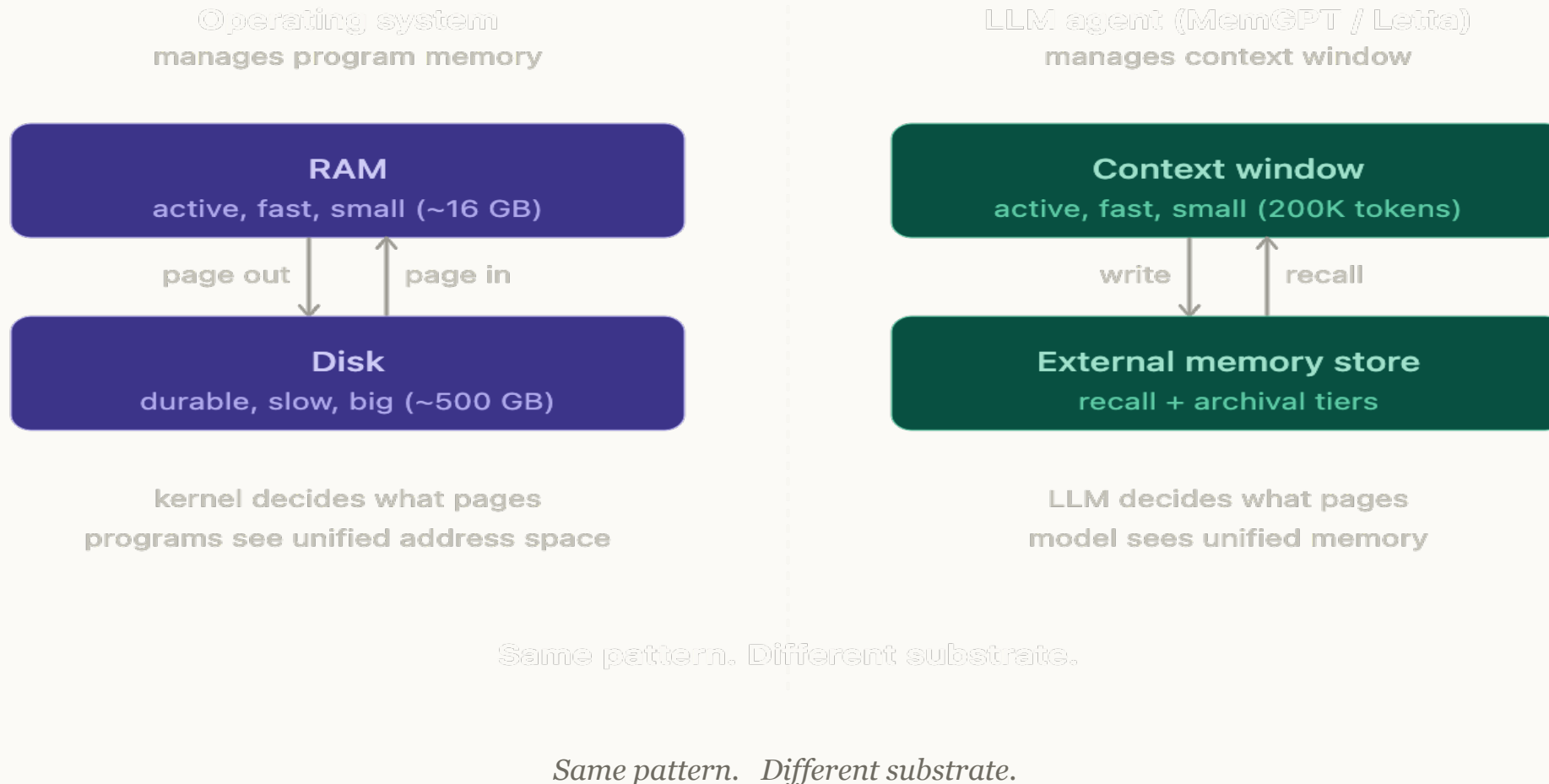
Temporal & portability.

- "Delhi" and "Bangalore" stored with equal weight — both still true.
-
- File is local to one machine.
-
- No team sharing.

What if the LLM managed its own memory?

- RAM is the context window.
-
- Disk is the external store.
-
- The LLM pages memory in and out.

Virtual memory, ported.



Three tiers.

Core

— *always loaded*

Recall

— *searchable history*

Archival

— *cold storage*

MemGPT (2023) → Letta.

- The LLM calls memory functions directly.
- Agent runtime, not a service.
- Opinionated by design.

What if a separate system extracted facts?

- Memory layer alongside the conversation.
- Watches every turn.
- Extracts facts. Stores them.

The Delhi → Bangalore problem.

- Append fails.
-
- Two contradictory memories.
-
- Need write-time logic.

Four decisions at write time.

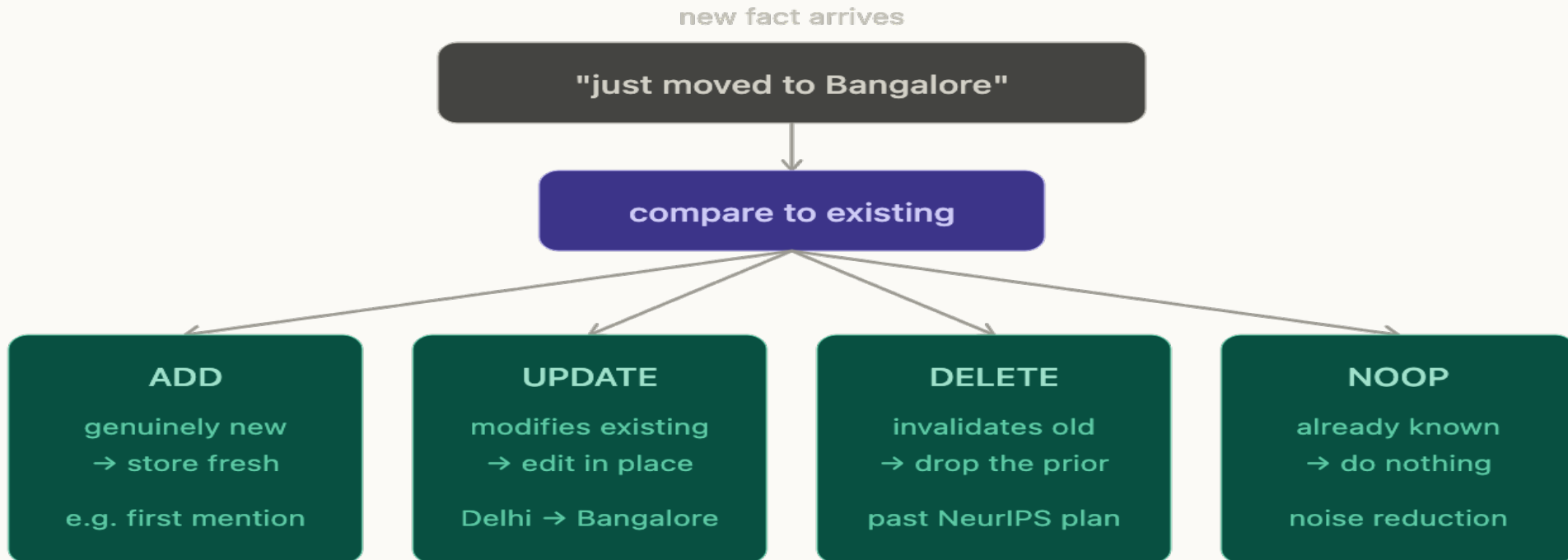
ADD — *genuinely new*

UPDATE — *modifies existing*

DELETE — *invalidates old*

NOOP — *already known*

ADD · UPDATE · DELETE · NOOP.



Write-time intelligence, not just write-time storage
one published approach — mem0 paper, ECAI 2025

Write-time intelligence, not just write-time storage.

One published approach.

— memo paper, ECAI 2025.

— LoCoMo 91.6% — vendor-reported.

— Other shapes exist: Letta, Zep.

Three conversations.

Three relationship types.

updates

— *new replaces old*

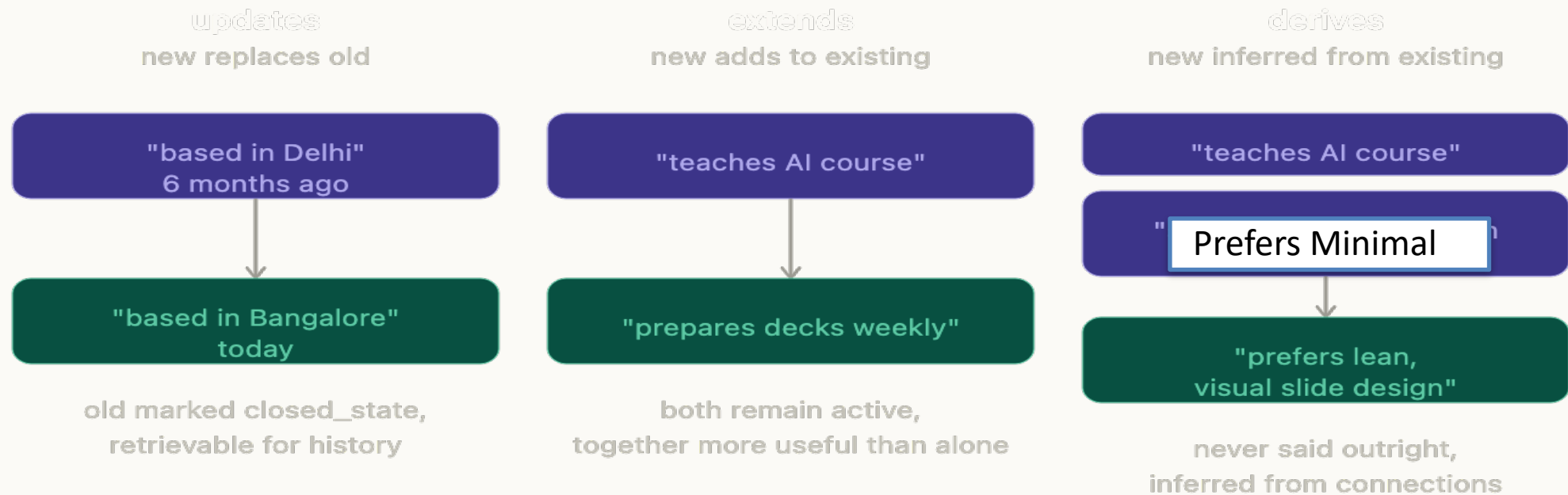
extends

— *new adds to existing*

derives

— *new inferred from existing*

Graph relationships.



The graph stores connections, not just facts
central in Supermemory's design

The graph stores connections, not just facts.

Supermemory's graph.

- Custom engine, not a generic graph DB.
- Traversal-based retrieval.
- Earns its complexity over months — not days.

Anthropic → AI safety lab.

JAN	joined Anthropic
OCT	left, started own lab

"Where did the user work last spring?"

Time signatures.

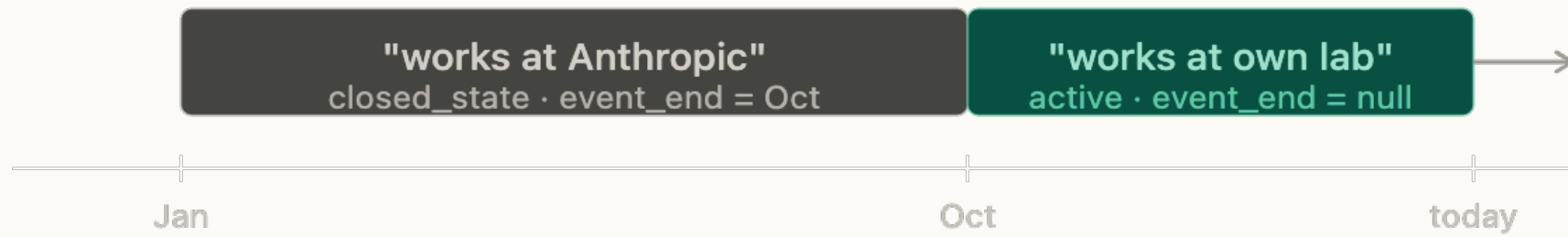
— event_start

— event_end

— Not metadata — structural.

Temporal state transition.

Every memory carries `event_start` and `event_end`
old memory isn't deleted — its window closes



"where does the user work?" → active memory
"where did the user work last spring?" → window overlap
"when did the user start the lab?" → `event_start` lookup

The old memory isn't deleted — its window closes.

Two implementations.

- memo temporal layer (May 2026).
-
- Zep / Graphiti — bi-temporal edges.
-
- LongMemEval 94.8% — vendor-reported.

The expiring exam.

"Exam tomorrow on transformer attention."

Six months later —

"What do I usually study around finals?"

First memory is now stale. Second still useful.

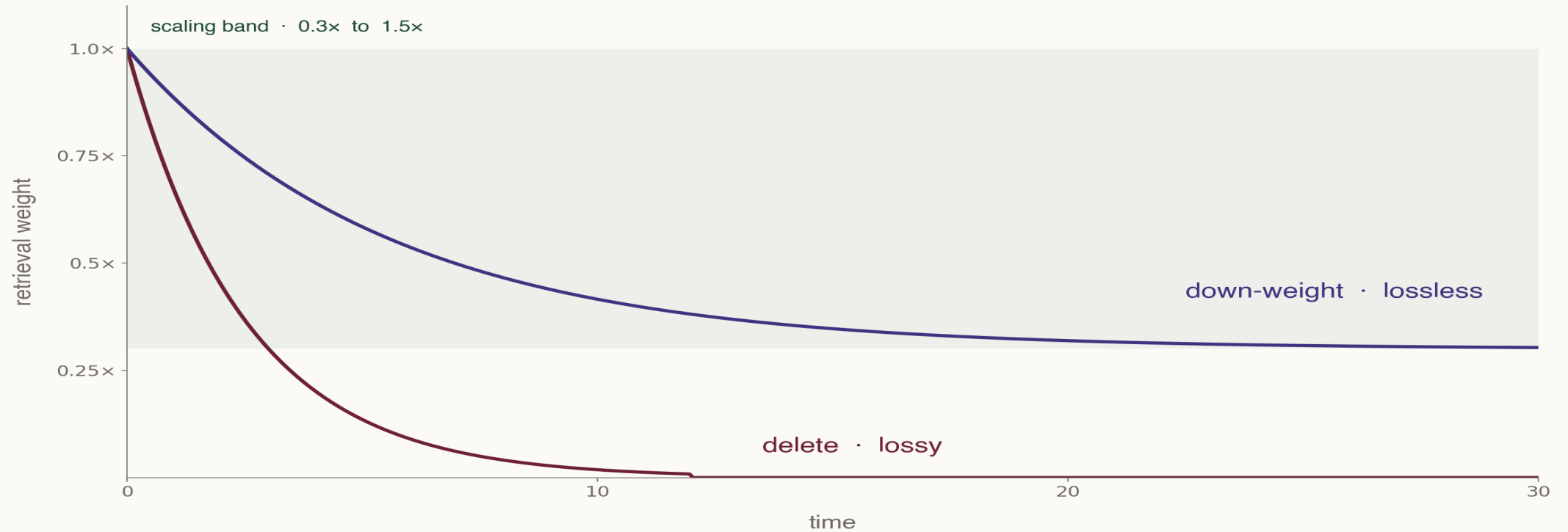
Delete or down-weight?

- Delete is lossy.
 - Down-weight is lossless.
 - Scaling band: $0.3\times$ to $1.5\times$.
-

Memory decay.

Memory decay — delete or down-weight

Ebbinghaus, 1885 · MemoryBank, 2023 · mem0 decay, 2026



Lower weight, not lost — recoverable when needed.

Old idea, new home.

1885	Ebbinghaus forgetting curve
------	-----------------------------

2023	MemoryBank
------	------------

2026	memo memory decay
------	-------------------

Three benchmarks.

LoCoMo — *Snap*

LongMemEval — *vendor-published*

ConvoMem — *Salesforce*

Vendor numbers.

memo

LoCoMo 91.6%

Supermemory

#1 across three

Zep

94.8% DMR

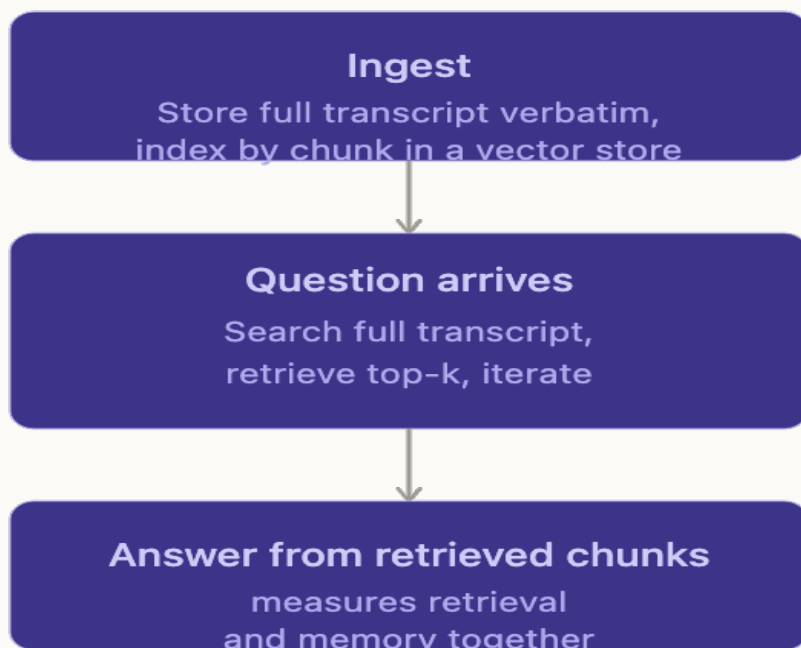
All numbers vendor-reported. Interpret accordingly.

But what's being measured?

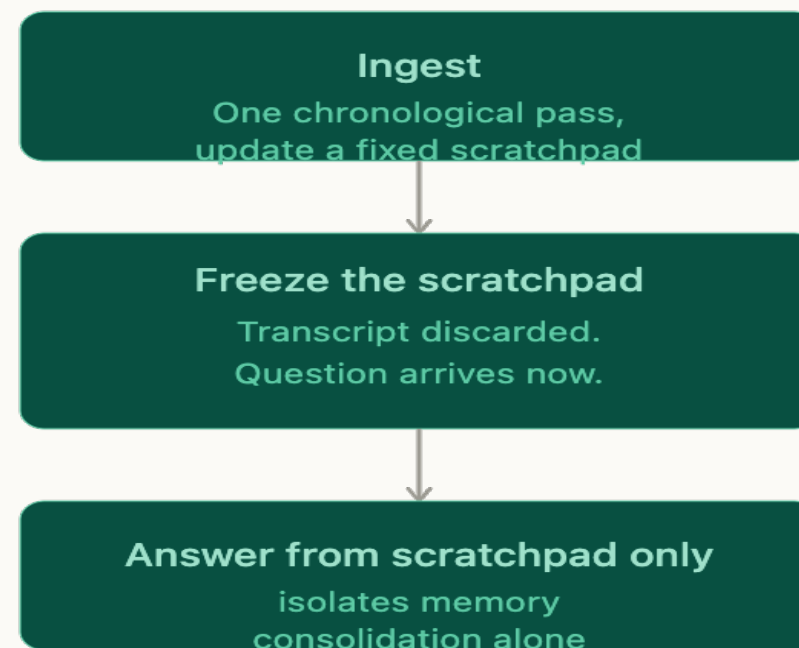
- Transcript still available at query time.
-
- The system can re-search.
-
- That's retrieval — not memory consolidation.

Standard benchmark vs. NoReplay setup.

Standard memory benchmark
transcript available at query time



NoReplay setup (Agarwal)
transcript discarded after ingest



Same task, different question. What does the score mean?

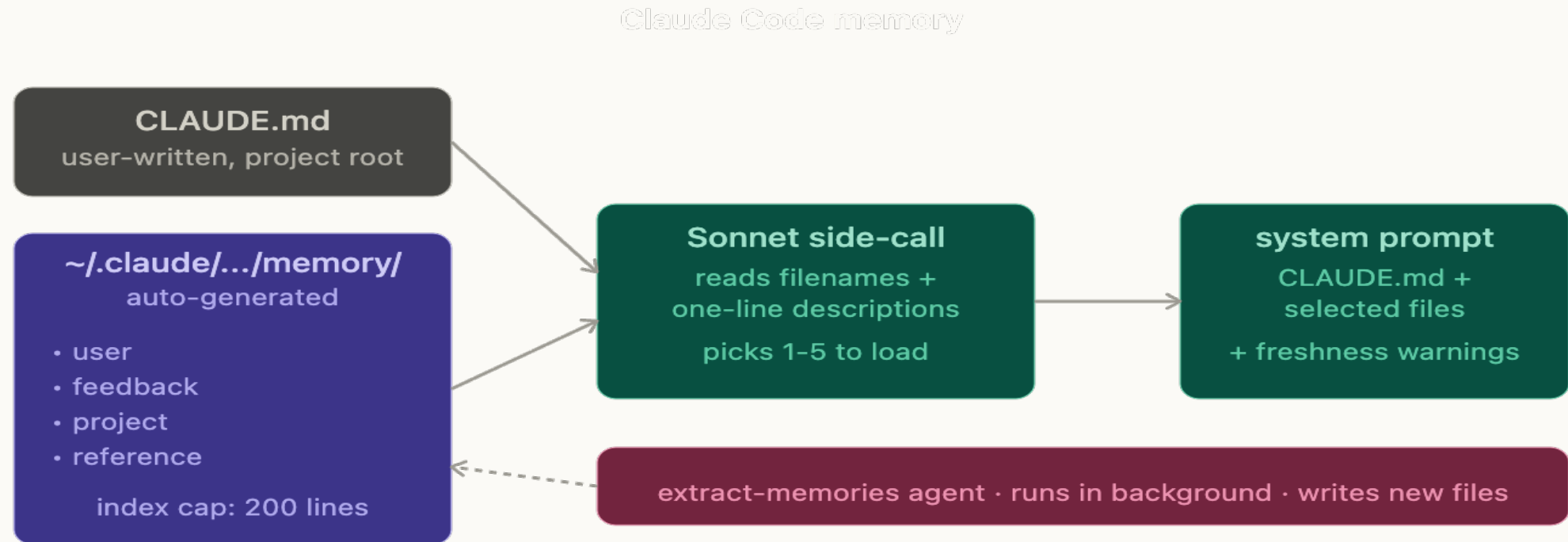
NoReplay. Agarwal et. al.

- One-pass ingest.
-
- Frozen scratchpad.
-
- Transcript discarded.

Claude Code.

- Two layers: CLAUDE.md + auto-memory.
-
- Four memory types.
-
- Sonnet side-call, not embeddings.

Claude Code memory architecture.



no embeddings · no vector store · LLM as relevance filter

No embeddings. No vector store. LLM as relevance filter.

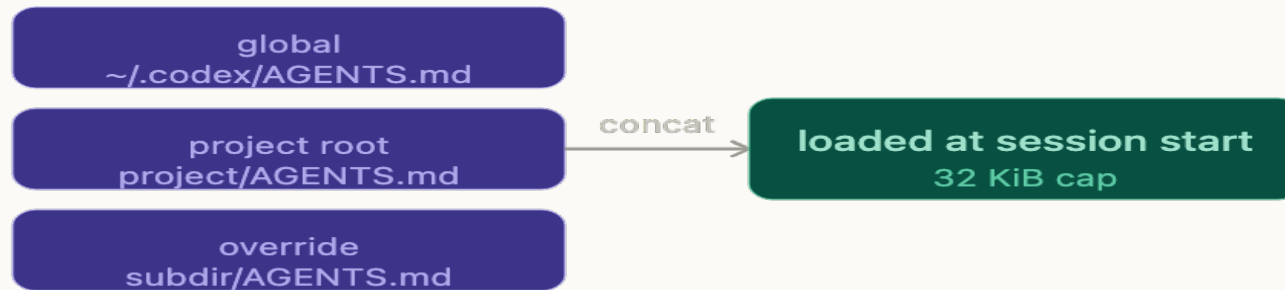
Codex.

- AGENTS.md — layered discovery.
- Memories — async between sessions.
- Grep over MEMORY.md, not vectors.

Codex memory architecture.

Codex memory: two pipelines

AGENTS.md (static)



Memories (async)



recall: session reads summary, greps MEMORY.md when needed

no embeddings · async writes only · 30-day TTL · EEA/UK/CH unavailable

Two pipelines: static AGENTS.md + async Memories.

Tradeoffs.

Letta

— *most rigorous OS pattern*

memo

— *most published research*

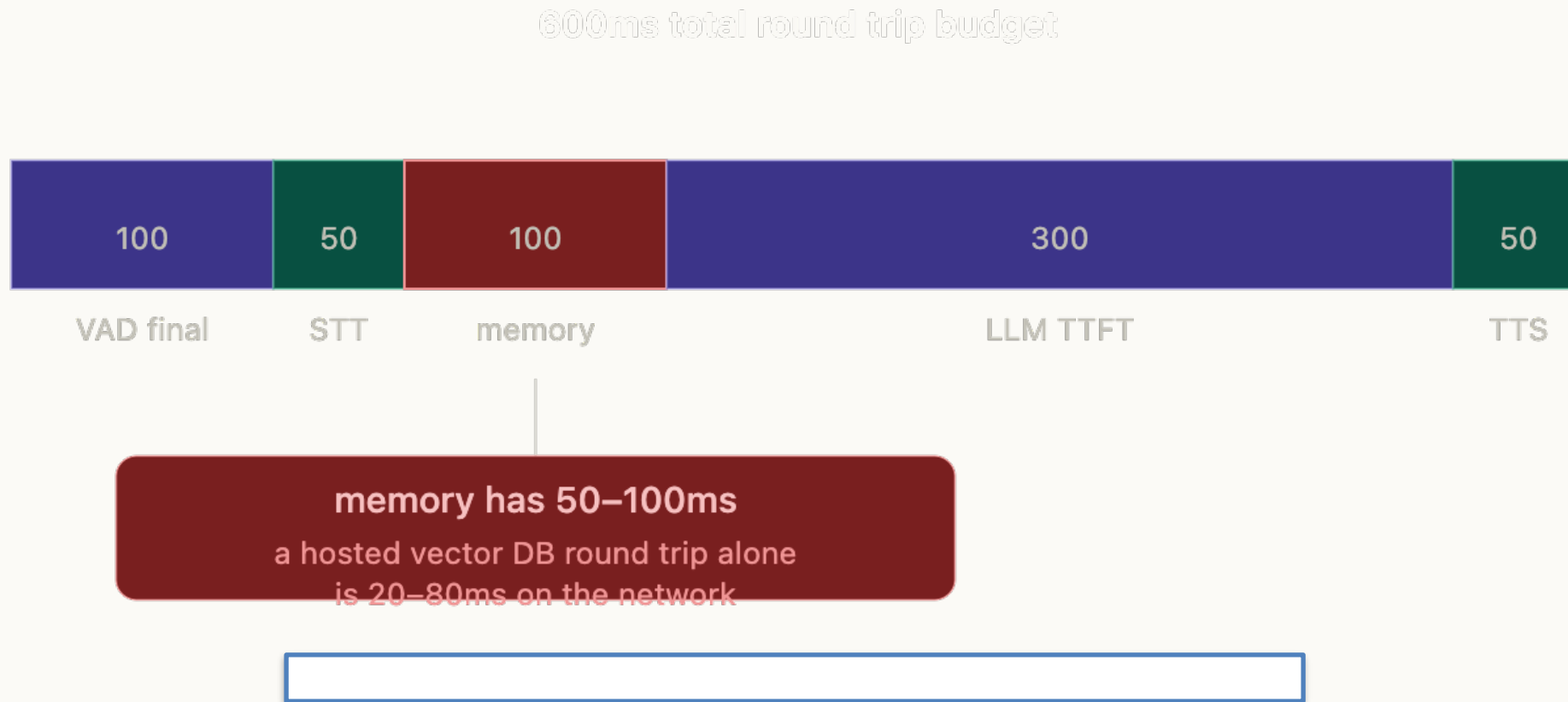
Supermemory

— *breadth across interfaces*

Voice.

- Total round trip: 500–800 ms.
-
- Memory budget: 50–100 ms.
-
- A network call alone takes about that.

Voice latency budget.



A 600 ms round trip leaves 50–100 ms for memory.

The inversion.

- Nothing expensive between turns.
 - Pre-load before the call.
 - Async writes after.
-

Three tiers.

Hot cache	1–5 ms	<i>blocking</i>
Background retrieval	50–150 ms	<i>async</i>
Async writes	—	<i>latency irrelevant</i>

Voice — three-tier stack.

Three tiers, defined by latency budget

Tier 1 — hot cache

1–5ms · blocking · in-process memory

pre-loaded before the call: user profile, last-call summary, open issues

Tier 2 — background retrieval

50–150ms · async between turns · staged for next turn

episodic search, fires every 3–5 turns or on topic shift

Tier 3 — async writes

latency irrelevant · runs after turns and after call

fact extraction, summarization, consolidation, conflict resolution

nothing expensive happens between
"user stops talking" and "agent starts replying"

Pre-loaded · background · async — defined by latency budget.

Failure modes.

- Cold start — anonymous caller.
 - Race condition — pre-load versus write-back.
 - Per-turn extraction cost.
-

What's still open.

- Cross-tool portability.
 - Privacy & consent.
 - Team memory.
 - A core-identity layer.
-

A five-layer future architecture.



Sensory

filters incoming information across modalities — text, voice, files, events

Short-term

active topics across reasoning, working scratchpad

Long-term

semantic + episodic memories with relationships, graph + temporal

Memory managers

background processes during idle: consolidation, pruning, reflection

Core

stable identity, evolves slowly, shapes how everything else is interpreted

One sketch — sensory · short-term · long-term · managers · core.

Five layers.

Sensory

— *filter incoming across modalities*

Short-term

— *active reasoning, scratchpad*

Long-term

— *semantic + episodic with relationships*

Memory managers

— *background consolidation, pruning*

Core

— *stable identity*

Takeaway.

- Memory is not solved.
- Infrastructure is mature.
- Semantics are still craft.
- Speed is set by what you prepared, not what you fetch.

Questions.

Take any of the thirteen parts.